Problem Set 3

Due January 30, 2026 at 11:59 PM Pacific

Turn in your problem set by Gradescope. Include any code you write in your solutions as well. Mark down how much time you spent on the homework set.

1. [10 Points.] Consider the position of a mobile robot x and a GPS measurement of that position y. Suppose these two random vectors are jointly Gaussian with parameters

$$\mu_x = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \Sigma_{xx} = \begin{bmatrix} 0.7 & 0.73 \\ 0.73 & 1.1 \end{bmatrix}, \quad \mu_y = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \Sigma_{yy} = \begin{bmatrix} 0.7 & 0.19 \\ 0.19 & 0.16 \end{bmatrix}, \quad \Sigma_{xy} = \begin{bmatrix} 0.63 & 0.23 \\ 0.72 & 0.31 \end{bmatrix}.$$

   Given a GPS position measurement $y = [0.27 \quad 0.62]^T$, find the posterior mean and covariance $(\mu_{x|y}, \Sigma_{x|y})$ of the rover position. Plot the following:

   - 95% error ellipse for the prior $\mathcal{N}(\mu_x, \Sigma_{xx})$
   - 95% error ellipse for the posterior $\mathcal{N}(\mu_{x|y}, \Sigma_{x|y})$
   - the mean of the posterior $\mu_{x|y}$
   - the GPS measurement y

2. [10 Points.] In this problem, you will prove that $E[(X_t - \mu_{t|t})W_t^T \mid Y_{1:t}] = 0$, which will be useful in the derivation of the Kalman Filter in lecture.

   (a) [5 Points.] Show that for three (not necessarily Gaussian) random variables (X, Y, W), if $p(x, y, w) = p(x, y)p(w)$ then $\text{Cov}(X, W \mid Y) = 0$. Hint: write out the integral for $\text{Cov}(\cdot)$ and manipulate the distribution under the integral.

   (b) [5 Points.] Use the above result to prove $E[(X_t - \mu_{t|t})W_t^T \mid Y_{1:t}] = 0$.

3. [10 Points.] Consider a quadrotor drone flying at a constant altitude. Let $p_t \in \mathbb{R}^2$ be its position, and $s_t \in \mathbb{R}^2$ its velocity. Let's model its dynamics as a discrete time double integrator perturbed by noise,

$$P_{t+1} = P_t + \delta t S_t \tag{1}$$
$$S_{t+1} = S_t + \delta t u_t + W_t, \tag{2}$$

   where $u_t \in \mathbb{R}^2$ is the acceleration commanded by a remote human pilot, and δt is the discrete time step. The drone makes position measurements with GPS according to the noisy measurement model

$$Y_t = P_t + V_t. \tag{3}$$

   The processes W_t, V_t are Gaussian white noise with covariances Q and R, respectively, uncorrelated with each other and uncorrelated with the initial state. Let $\delta t = 1s$, $Q = I_2 m^2/s^2$, $R = 9\text{I}_2 m^2$, $u_t = -2.5[\cos(0.05t) \quad \sin(0.05t)]^T m/s^2$.

   (a) [5 Points.] Find the system matrices A, B, C, in terms of the given parameters.

   (b) [5 Points.] Simulate the trajectory of the drone (in Python) as well as the measurement process. Let the initial position and velocity be $p_0 = [1000m \quad 0m]^T$, and $v_0 = [0m/s \quad 50m/s]^T$, respectively. Attach plots of a few example trajectories along with the noisy GPS position measurements.

   Hint: The Python command `numpy.random.randn` will come in handy for producing the noise signals.

4. [15 Points.] Now you will run a Kalman filter to estimate the drone's position and velocity.

   (a) [5 Points.] Due to the noise only appearing in the velocity equation, we must change one of the Kalman filter equations slightly to accomodate this system. What is the new expression?

(b) [9 Points.] Suppose your initial guess about the state of the drone has a mean $\mu_0 = [1500m \quad 100m \quad 0m/s \quad 55m/s]^T$, and covariance matrix $\Sigma_0 = [250000\text{I}m^2 \quad 0; \quad 0 \quad \text{I}m^2/s^2]$ (i.e. we are pretty sure about its velocity, but not its position). Use the Kalman filter to find your estimated state trajectory. Use your error ellipse script from PS2 to plot the means and error ellipses of the position estimate over the top of the true trajectory.

(c) [1 Point.] Similarly, plot the mean and error ellipses of the velocity along the trajectory of the drone (plotting them as vectors emanating from the drone's position, with the ellipse centered at the tip of the vector). We have essentially used the Kalman filter as a very fancy numerical differentiator to find velocity from GPS.

5. [5 Points.] Consider the same model of the drone, but we only have a velocity sensor with the measurement equation

$$Y_t = S_t + V_t.$$

The parameters are the same, and the initial state estimate is $\mu_0 = [1000m \quad 0m \quad 0m/s \quad 50m/s]^T$ with covariance $\Sigma_0 = [\text{I}m^2 \quad 0; \quad 0 \quad \text{I}m^2/s^2]$ (i.e. we are pretty sure about the initial state). Run the Kalman filter to estimate the position and velocity in this case. Plot them in the same way as before. Here we have essentially used the Kalman filter as a very fancy numerical integrator to get position from velocity measurements.[1]

[1] This is called "dead reckoning," and as you can see, it cannot provide good position estimates for very long.

1. [10 Points.] Consider the position of a mobile robot x and a GPS measurement of that position y. Suppose these two random vectors are jointly Gaussian with parameters

$$\mu_x = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \Sigma_{xx} = \begin{bmatrix} 0.7 & 0.73 \\ 0.73 & 1.1 \end{bmatrix}, \quad \mu_y = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \Sigma_{yy} = \begin{bmatrix} 0.7 & 0.19 \\ 0.19 & 0.16 \end{bmatrix}, \quad \Sigma_{xy} = \begin{bmatrix} 0.63 & 0.23 \\ 0.72 & 0.31 \end{bmatrix}.$$

Given a GPS position measurement $y = [0.27 \quad 0.62]^T$, find the posterior mean and covariance $(\mu_{x|y}, \Sigma_{x|y})$ of the rover position. Plot the following:

- 95% error ellipse for the prior $\mathcal{N}(\mu_x, \Sigma_{xx})$
- 95% error ellipse for the posterior $\mathcal{N}(\mu_{x|y}, \Sigma_{x|y})$
- the mean of the posterior $\mu_{x|y}$
- the GPS measurement y

$$\Sigma_{yy} = \begin{bmatrix} 0.7 & 0.19 \\ 0.19 & 0.16 \end{bmatrix} \downarrow \quad \Sigma_{yy}^{-1} = \begin{bmatrix} 2.1080 & -2.5033 \\ -2.5033 & 9.2227 \end{bmatrix}$$

$$\mu_{x|y} = \mu_x + \Sigma_{xy}\Sigma_y^{-1}(y - \mu_y)$$

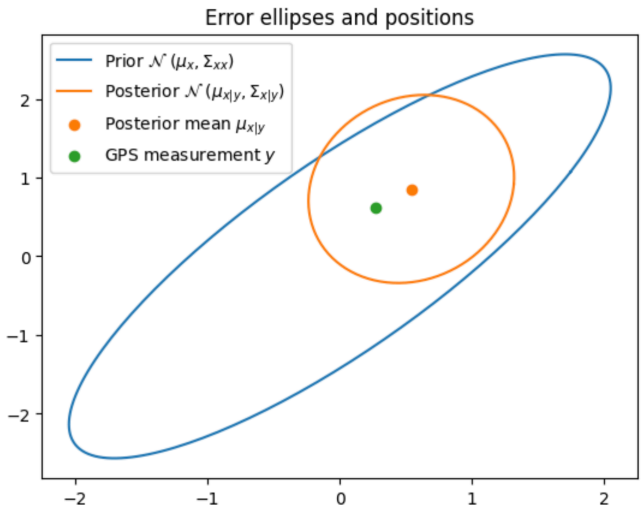
$$= \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0.63 & 0.23 \\ 0.72 & 0.31 \end{bmatrix} \begin{bmatrix} 2.1080 & -2.5033 \\ -2.5033 & 9.2227 \end{bmatrix} \left(\begin{bmatrix} 0.27 \\ 0.62 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \end{bmatrix} \right) \qquad \mu_{x|y} = \begin{bmatrix} 0.5405 \\ 0.8554 \end{bmatrix}$$

$$\Sigma_{yx} = \Sigma_{xy}^T = \begin{bmatrix} 0.63 & 0.23 \\ 0.72 & 0.31 \end{bmatrix}^T = \begin{bmatrix} 0.63 & 0.72 \\ 0.23 & 0.31 \end{bmatrix}$$

$$\Sigma_{x|y} = \Sigma_x - \Sigma_{xy}\Sigma_y^{-1}\Sigma_{yx}$$

$$= \begin{bmatrix} 0.7 & 0.73 \\ 0.73 & 1.1 \end{bmatrix} - \begin{bmatrix} 0.63 & 0.23 \\ 0.72 & 0.31 \end{bmatrix} \begin{bmatrix} 2.1080 & -2.5033 \\ -2.5033 & 9.2227 \end{bmatrix} \begin{bmatrix} 0.63 & 0.72 \\ 0.23 & 0.31 \end{bmatrix}$$

$$\Sigma_{x|y} = \begin{bmatrix} 0.1009 & 0.0197 \\ 0.0197 & 0.2384 \end{bmatrix}$$



2. [10 Points.] In this problem, you will prove that $E[(X_t - \mu_{t|t})W_t^T \mid Y_{1:t}] = 0$, which will be useful in the derivation of the Kalman Filter in lecture.

   (a) [5 Points.] Show that for three (not necessarily Gaussian) random variables (X, Y, W), if $p(x, y, w) = p(x, y)p(w)$ then $\text{Cov}(X, W \mid Y) = 0$. Hint: write out the integral for $\text{Cov}(\cdot)$ and manipulate the distribution under the integral.

   (b) [5 Points.] Use the above result to prove $E[(X_t - \mu_{t|t})W_t^T \mid Y_{1:t}] = 0$.

$$p(x,w|y) = \frac{p(x,w,y)}{p(y)}$$

$$= \frac{p(x,y)p(w)}{p(y)} = p(x|y)p(w)$$

$p(x|y)$

a) $$\text{cov}(X,W|Y) = E\left[(X-\mu_{x|y})(W-\mu_{w|y})^T\right]$$

$$= \int_{x_i}\int_{w_i}(x_i-\mu_{x|y})(w_i-\mu_{w|y})\,p(x_i,w_i\mid y_i)\,dw_i\,dx_i$$

$$= \int_{x_i}\int_{w_i}(x_i-\mu_{x|y})(w_i-\mu_{w|y})\,p(x_i|y_i)p(w_i)\,dw_i\,dx_i$$

$$= \int_{x_i}(x_i-\mu_{x|y})\,p(x_i|y_i)\,dx_i \int_{w_i}(w_i-\mu_{w|y})\,p(w_i)\,dw_i$$

(1)

(1) $$\int_{x_i} x_i\,p(x_i|y_i)\,dx_i - \mu_{x|y}\int_{x_i}p(x_i|y_i)\,dx_i = \mu_{x|y} - \mu_{x|y} = 0$$

$\mu_{x|y}$

a: $$\text{Cov}(X,W|Y) = 0$$

b) $$\text{cov}(X,W|Y) = 0 = E\left[(X-\mu_{x|y})(W-\mu_{w|y})^T \mid Y\right]$$

$$= E\left[(X-\mu_{x|y})W^T|Y\right] - E\left[(X-\mu_{x|y})\mu_{w|y}^T\mid Y\right]$$

$$= E\left[(X-\mu_{x|y})W^T|Y\right] - E\left[X\mu_{w|y}^T\mid Y\right] + E\left[\mu_{x|y}\mu_{w|y}^T\right]$$

$$= E\left[(X-\mu_{x|y})W^T|Y\right] - E\left[X|Y\right]\mu_{w|y}^T + \mu_{x|y}\mu_{w|y}^T$$

↓ conditional expectation

$$= E\left[(X-\mu_{x|y})W^T|Y\right] - \mu_{x|y}\mu_{w|y}^T + \mu_{x|y}\mu_{w|y}^T$$

$$0 = E\left[(X-\mu_{x|y})W^T|Y\right]$$

$$= E[XW^T|Y] - E[\mu_{x|y} W^T] - E[X\mu_{w|y}^T|Y] + E[\mu_{x|y}\mu_{w|y}^T|Y] = 0$$

$$= E[XW^T|Y] - \mu_{x|y} E[W^T|Y] - E[X|Y]\mu_{w|y}^T + \mu_{x|y}\mu_{w|y}^T$$

$$E[XW^T|Y] - \mu_{x|y} E[W^T|Y] - \mu_{x|y}\mu_{w|y}^T + \mu_{x|y}\mu_{x|y}^T$$

$$E[XW^T|Y]$$

3. [10 Points.] Consider a quadrotor drone flying at a constant altitude. Let $p_t \in \mathbb{R}^2$ be its position, and $s_t \in \mathbb{R}^2$ its velocity. Let's model its dynamics as a discrete time double integrator perturbed by noise,

$$P_{t+1} = P_t + \delta t S_t \tag{1}$$
$$S_{t+1} = S_t + \delta t u_t + W_t, \tag{2}$$

where $u_t \in \mathbb{R}^2$ is the acceleration commanded by a remote human pilot, and δt is the discrete time step. The drone makes position measurements with GPS according to the noisy measurement model

$$Y_t = P_t + V_t. \tag{3}$$

The processes W_t, V_t are Gaussian white noise with covariances Q and R, respectively, uncorrelated with each other and uncorrelated with the initial state. Let $\delta t = 1s$, $Q = I_2 m^2/s^2$, $R = 9I_2 m^2$, $u_t = -2.5[\cos(0.05t) \;\; \sin(0.05t)]^T m/s^2$.

(a) [5 Points.] Find the system matrices A, B, C, in terms of the given parameters.

(b) [5 Points.] Simulate the trajectory of the drone (in Python) as well as the measurement process. Let the initial position and velocity be $p_0 = [1000m \;\; 0m]^T$, and $v_0 = [0m/s \;\; 50m/s]^T$, respectively. Attach plots of a few example trajectories along with the noisy GPS position measurements.

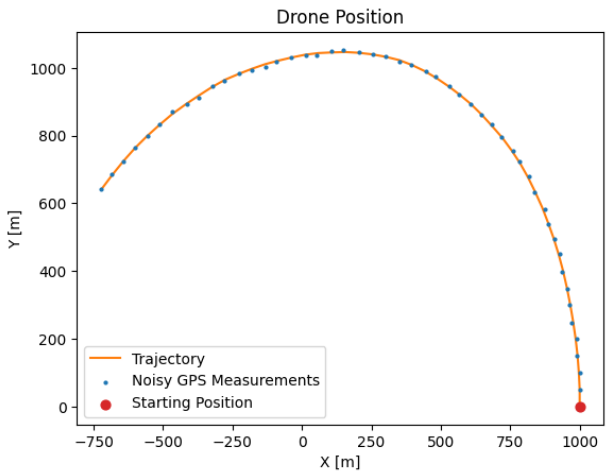
Hint: The Python command `numpy.random.randn` will come in handy for producing the noise signals.

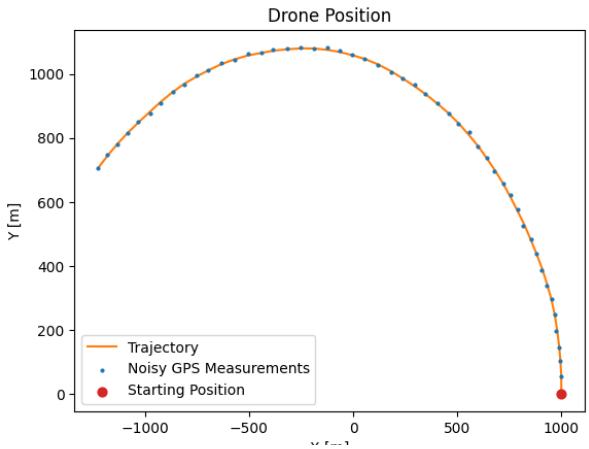
$$\begin{bmatrix} p \\ s \end{bmatrix}_{t+1} = \begin{bmatrix} 1 & \delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} p \\ s \end{bmatrix}_t + \begin{bmatrix} 0 \\ \delta t \end{bmatrix} u_t + w_t$$

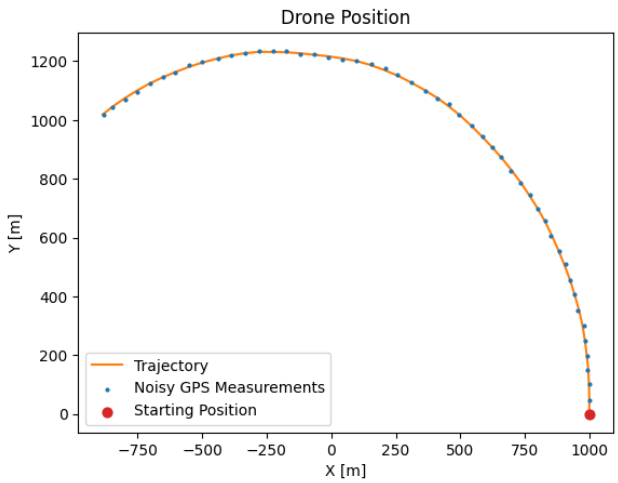
$$A = \begin{bmatrix} 1 & \delta t \\ 0 & 1 \end{bmatrix} \qquad A = \begin{bmatrix} I_2 & I_2 \\ 0 & I_2 \end{bmatrix}$$

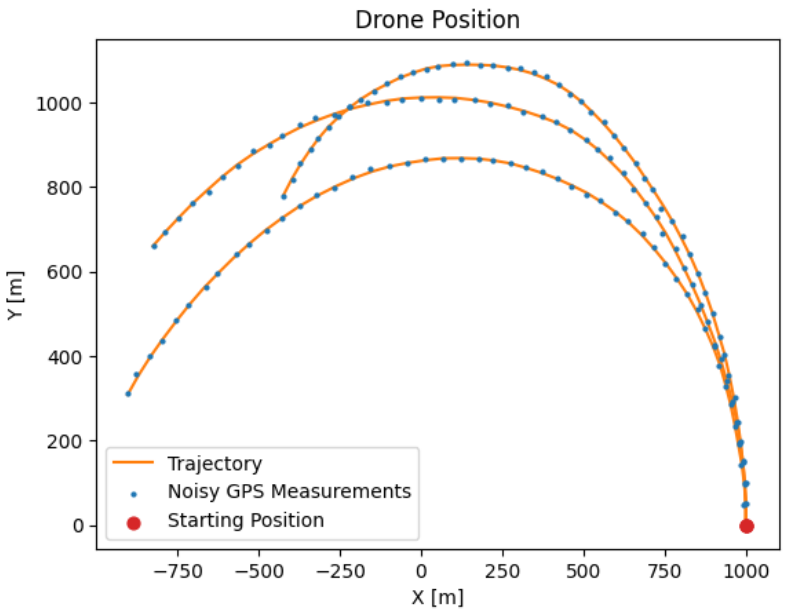
$$B = \begin{bmatrix} 0 \\ \delta t \end{bmatrix} \qquad B = \begin{bmatrix} 0 \\ I_2 \end{bmatrix}$$

$$y_t = [1 \;\; 0] \begin{bmatrix} p \\ s \end{bmatrix}_t + v_t \qquad C = [I_2 \;\; 0]$$









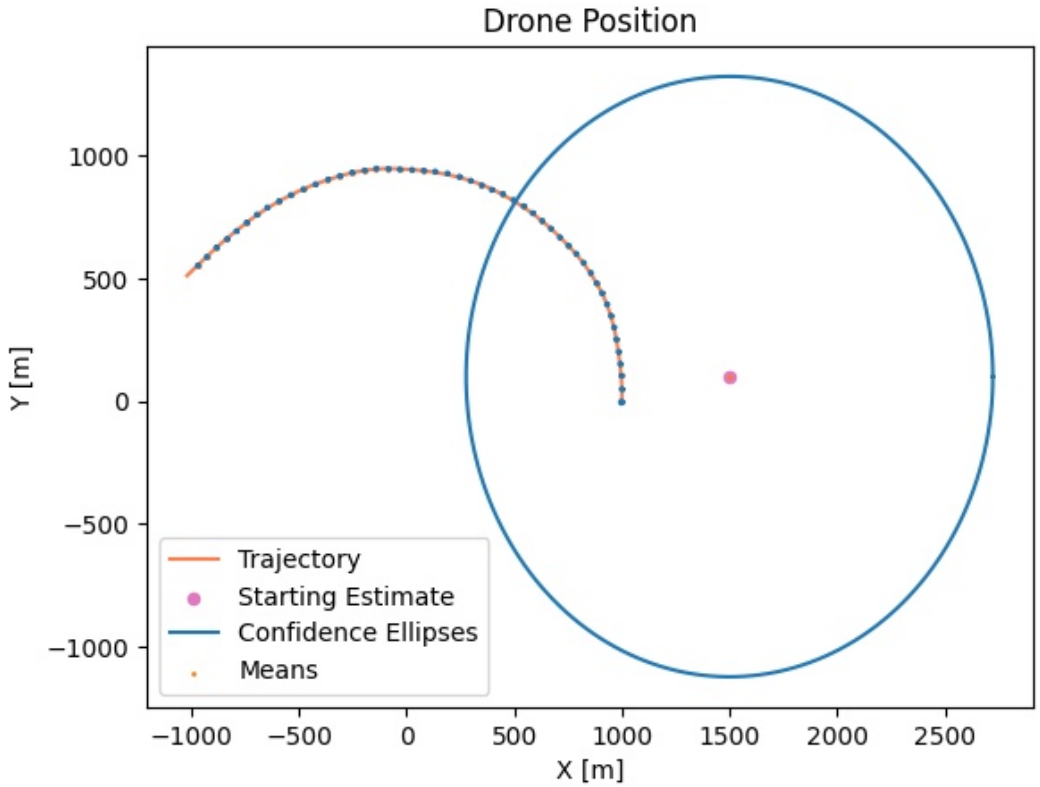
4. [15 Points.] Now you will run a Kalman filter to estimate the drone's position and velocity.

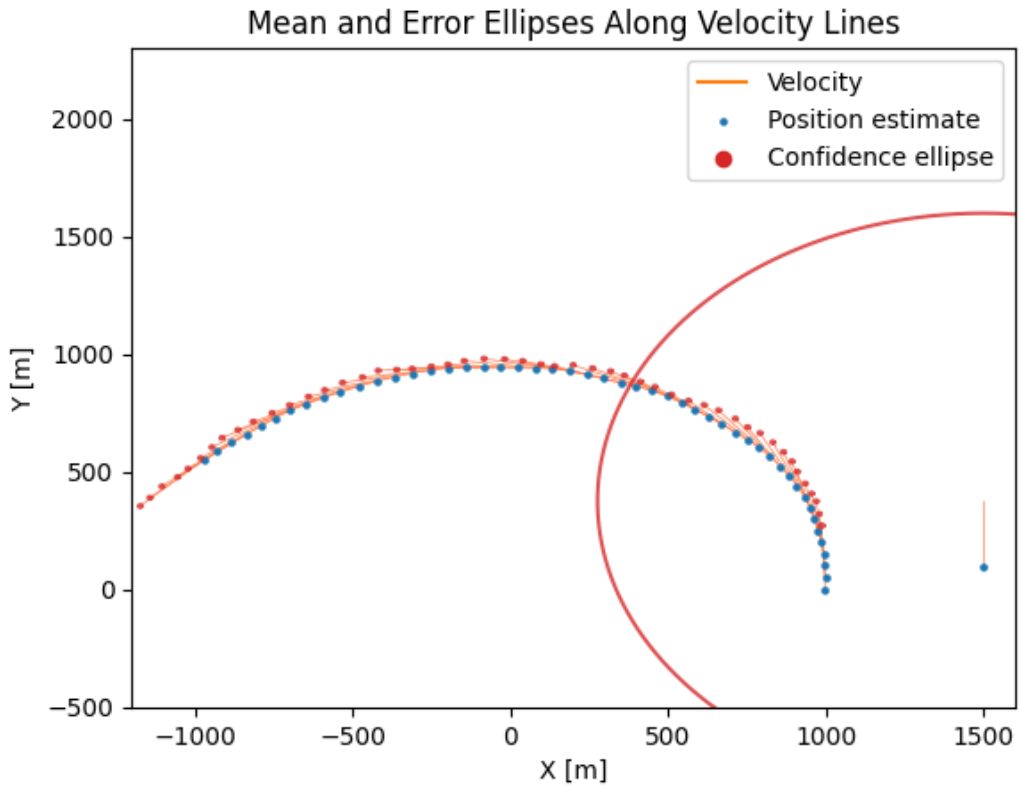
   (a) [5 Points.] Due to the noise only appearing in the velocity equation, we must change one of the Kalman filter equations slightly to accomodate this system. What is the new expression?

$$Q = \begin{bmatrix} 0_2 & 0_2 \\ 0_2 & I_2 \end{bmatrix}$$

   (b) [9 Points.] Suppose your initial guess about the state of the drone has a mean $\mu_0 = [1500m \quad 100m \quad 0m/s \quad 55m/s]^T$, and covariance matrix $\Sigma_0 = [250000Im^2 \quad 0; \quad 0 \quad Im^2/s^2]$ (i.e. we are pretty sure about its velocity, but not its position). Use the Kalman filter to find your estimated state trajectory. Use your error ellipse script from PS2 to plot the means and error ellipses of the position estimate over the top of the true trajectory.

   (c) [1 Point.] Similarly, plot the mean and error ellipses of the velocity along the trajectory of the drone (plotting them as vectors emanating from the drone's position, with the ellipse centered at the tip of the vector). We have essentially used the Kalman filter as a very fancy numerical differentiator to find velocity from GPS.



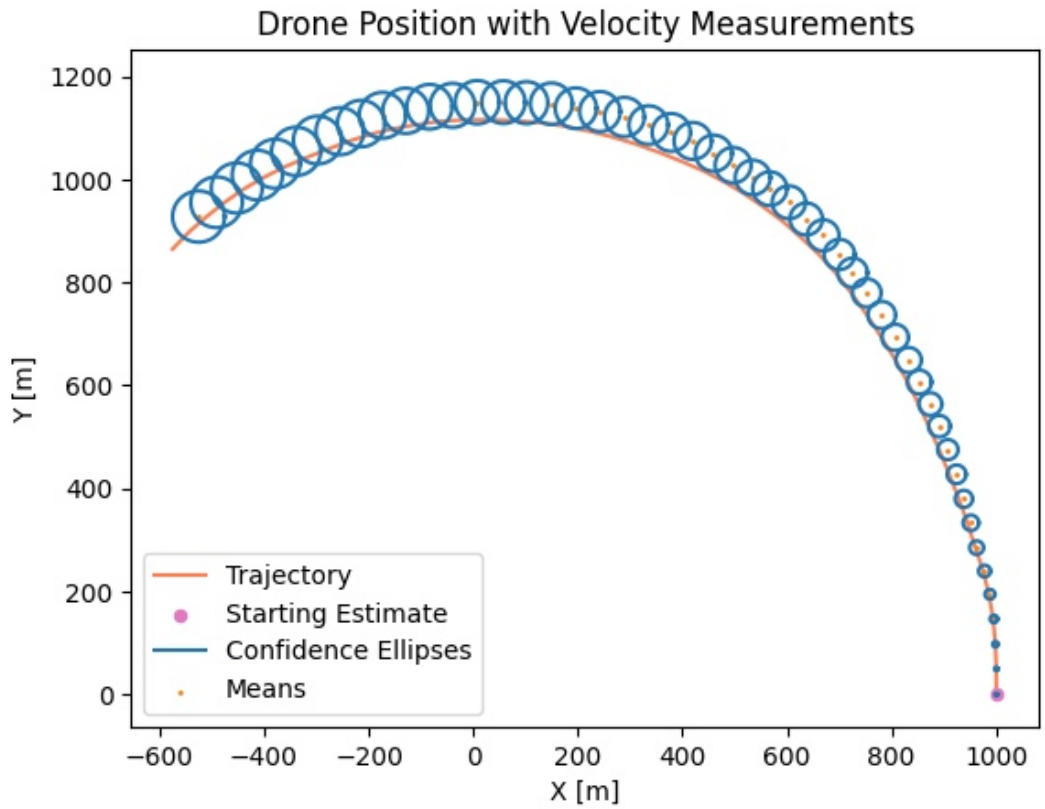


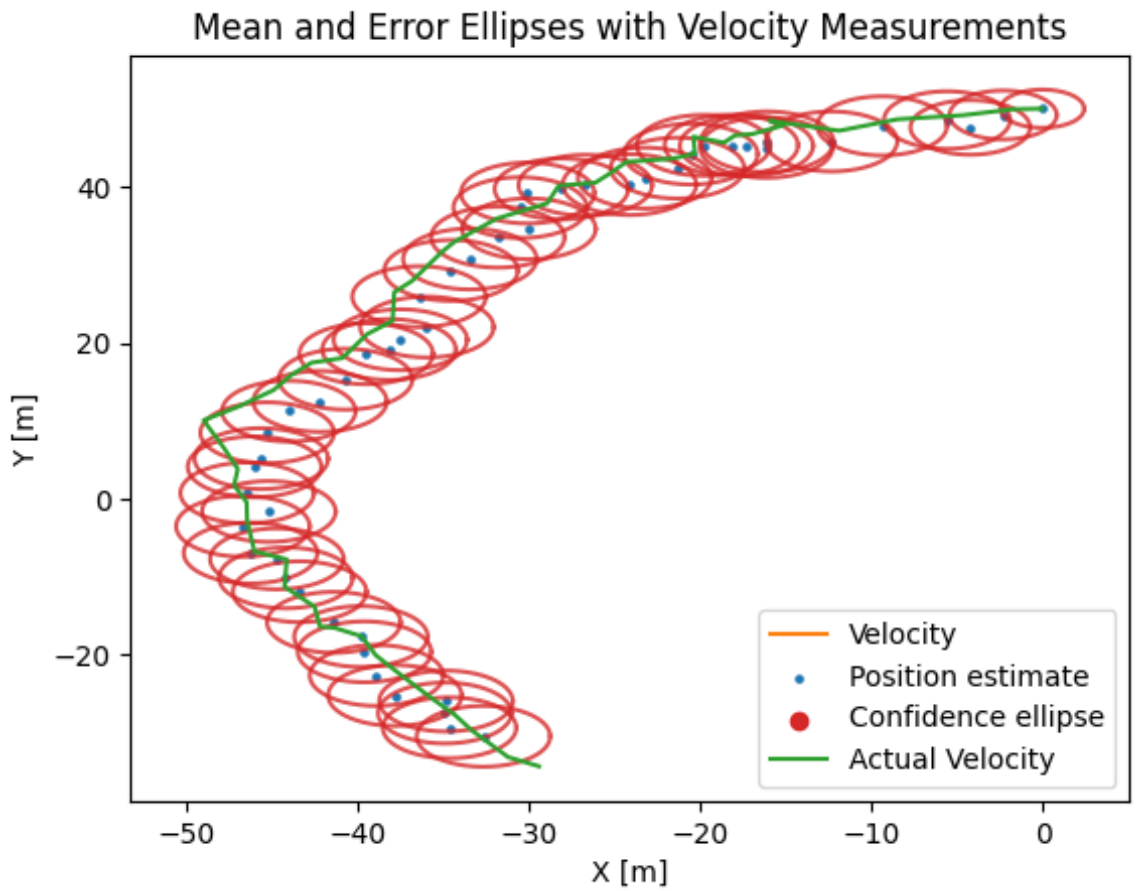
5. [5 Points.] Consider the same model of the drone, but we only have a velocity sensor with the measurement equation

$$Y_t = S_t + V_t.$$

The parameters are the same, and the initial state estimate is $\mu_0 = [1000m \quad 0m \quad 0m/s \quad 50m/s]^T$ with covariance $\Sigma_0 = [Im^2 \quad 0; \quad 0 \quad Im^2/s^2]$ (i.e. we are pretty sure about the initial state). Run the Kalman filter to estimate the position and velocity in this case. Plot them in the same way as before. Here we have essentially used the Kalman filter as a very fancy numerical integrator to get position from velocity measurements.[1]

$$y = [0 \; I_2] \begin{bmatrix} p \\ s \end{bmatrix}$$





In [1]:
```
import numpy as np
from numpy.random import randn
from scipy.linalg import sqrtm, inv
import matplotlib.pyplot as plt
```

In [2]:
```
def epsilon_and_nu(P: float, Sigma: np.ndarray):
    nu = 1/ (2 * np.pi * np.sqrt(np.linalg.det(Sigma)))
    epsilon = (1-P) * nu

    return epsilon, nu

def in_error_ellipse(x: np.ndarray, mu: np.ndarray, Sigma: np.ndarray, P=0.9
    epsilon, nu = epsilon_and_nu(P, Sigma)
    rhs = -2*np.log(epsilon / nu)   # = -2 log(1-P)
    lhs = (x - mu).T @ np.linalg.inv(Sigma) @ (x - mu)
    return lhs <= rhs

def plot_gaussian(mu, Sigma, N, show=True):
    x_arr = randn(N,2) @ sqrtm(Sigma) + mu

    x_in = np.array([x for x in x_arr if in_error_ellipse(x, mu, Sigma)])
    x_out = np.array([x for x in x_arr if not in_error_ellipse(x, mu, Sigma)

    e, _ = epsilon_and_nu(0.95, Sigma)
    print(f"Percentage of points in ellipse {len(x_in) / N * 100}")
    plt.scatter(x_in[:,0], x_in[:,1], marker='.')
    plt.scatter(x_out[:,0], x_out[:,1], marker='.')
    plt.title("95% Confidence Error Ellipse")
    if show:
        plt.show()

def get_ellipse_points(mu: np.ndarray, Sigma: np.ndarray, P=0.95, N=200):
    t = np.linspace(0, 2*np.pi, N)
    circle = np.vstack((np.cos(t), np.sin(t)))
    rhs = -2*np.log(1-P)
    ellipse = np.sqrt(rhs) * circle
    scaled_ellipse = sqrtm(Sigma) @ ellipse # circle -> ellipse
    return scaled_ellipse + mu.reshape(2,1)
```

Problem 1

Finding mu_x_y

In [3]:
```
mu_x = np.array([0, 0]).reshape((2,1))
mu_y = np.array([0, 0]).reshape((2,1))

Sigma_xx = np.array([
    [0.7, 0.73],
    [0.73, 1.1]])
Sigma_yy = np.array([
    [0.7, 0.19],
```

```
    [0.19, 0.16]])
Sigma_xy = np.array([
    [0.63, 0.23],
    [0.72, 0.31]])
Sigma_yy_inv = inv(Sigma_yy)
Sigma_yx = Sigma_xy.T

y = np.array([0.27, 0.62]).reshape((2,1))

print(Sigma_yy_inv)
print(Sigma_yx)
```

```
[[ 2.10803689 -2.50329381]
 [-2.50329381  9.2226614 ]]
[[0.63 0.72]
 [0.23 0.31]]
```

In [4]:
```
mu_x_y = mu_x + Sigma_xy @ Sigma_yy_inv @ (y - mu_y)
Sigma_x_y = Sigma_xx - Sigma_xy @ Sigma_yy_inv @ Sigma_yx

print(mu_x_y)
print(Sigma_x_y)
```

```
[[0.54048748]
 [0.85540184]]
[[0.10089592 0.01965744]
 [0.01965744 0.23836627]]
```

Plotting

In [5]:
```
plt.plot(*get_ellipse_points(mu_x, Sigma_xx), color='tab:blue', label=r"Pric

plt.plot(*get_ellipse_points(mu_x_y, Sigma_x_y), color='tab:orange', label=r
plt.scatter(*mu_x_y, color='tab:orange', label=r"Posterior mean $\mu_{x|y}$"

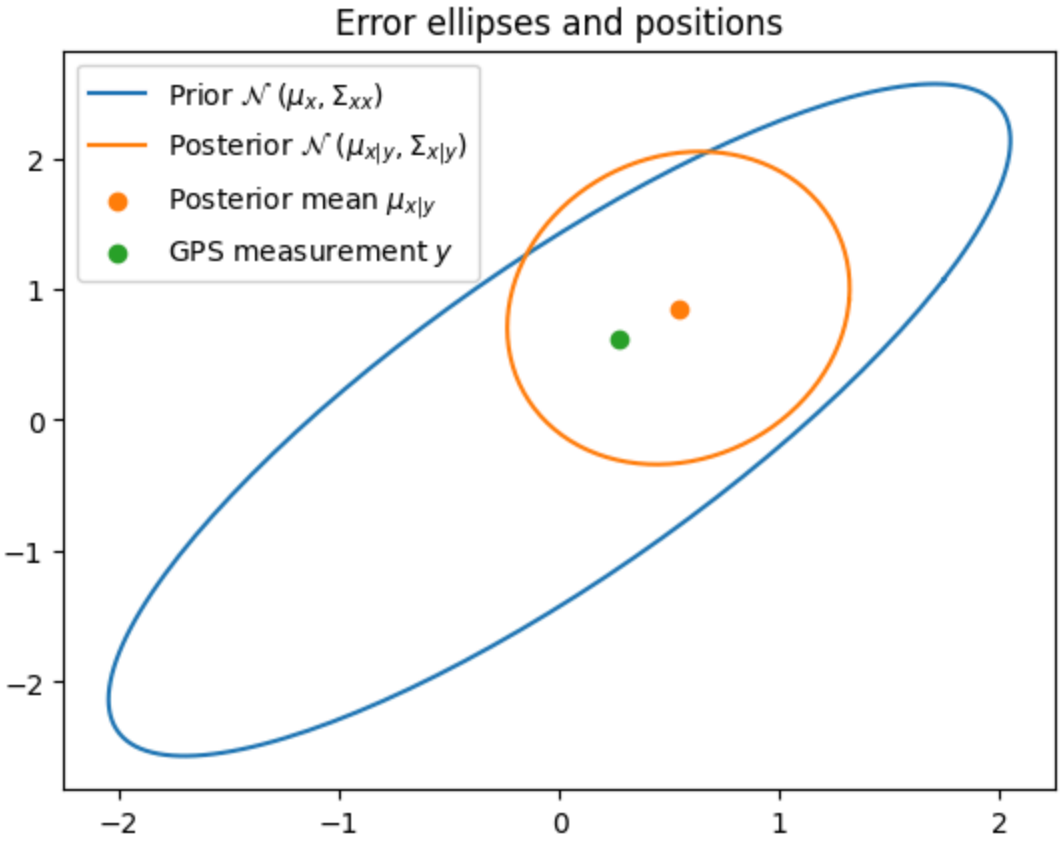
plt.scatter(*y, color='tab:green', label="GPS measurement $y$")

plt.legend()

plt.title("Error ellipses and positions")

plt.show()

# plt.scatter(*mu_x, color='orange')
# plot_gaussian(mu_x_y, Sigma_x_y, 1000, )
```



Problem 3

Creating trajectories

In [6]:
```
dt = 1
Q = np.zeros((4,4))
Q[2:4, 2:4] = np.eye(2)

R = 9*np.eye(2)

def u_input(t):
    return -2.5 * np.array([
        np.cos(0.05*t), np.sin(0.05*t)
    ])
```

In [7]:
```
A = np.zeros((4,4))
A[0:2, 0:2] = np.eye(2)
A[0:2, 2:4] = dt*np.eye(2)
A[2:4, 2:4] = np.eye(2)

B = np.zeros((4,2))
B[2:4, 0:2] = np.eye(2)

C = np.zeros((2,4))
C[0:2, 0:2] = np.eye(2)
```

```
x0 = np.array([1000, 0, 0, 50])
```

In [8]:
```
def noise_W(Q, seed=None):
    if seed is not None:
        np.random.seed(seed)

    return np.random.multivariate_normal(
        mean=np.zeros(4),
        cov=Q
    )

def noise_V(R, seed=None):
    if seed is not None:
        np.random.seed(seed)

    return np.random.multivariate_normal(
        mean=np.zeros(2),
        cov=R
    )

print(noise_W(Q))
print(noise_V(R))
```

```
[ 0.          0.          0.45740725 -1.39150488]
[-6.66604998 -1.41067637]
```

In [9]:
```
N = 50 # Timesteps of 1 second

class Trajectory:
    def __init__(self, x, u, meas):
        self.x_arr = x
        self.u_arr = u
        self.meas_arr = meas

def one_trajectory(N: int = 50, show=True, return_results = True):

    meas_arr = np.zeros((N,2))
    u_arr = np.zeros((N,2))
    x_arr = np.zeros((N,4))


    x_arr[0] = x0
    meas_arr[0] = C @ x0 + noise_V(R)

    for t in range(N-1):
        u_arr[t] = u_input(t).T

        x_arr[t+1] = A @ x_arr[t] + B @ u_arr[t] + noise_W(Q)
        meas_arr[t+1] = C @ x_arr[t+1] + noise_V(R)

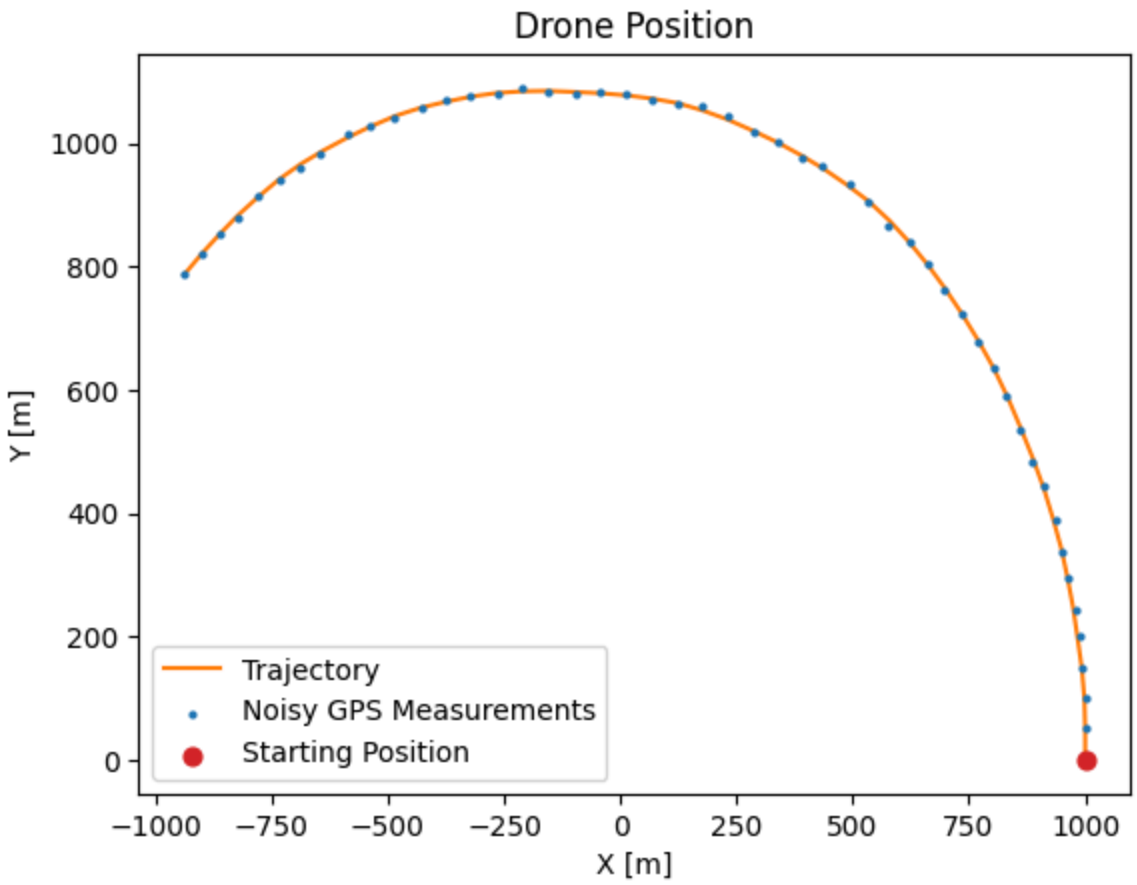


    if show:
        plt.plot(x_arr[:,0], x_arr[:,1], label="Trajectory", color="tab:orar
        plt.scatter(meas_arr[:,0], meas_arr[:,1], label="Noisy GPS Measureme
```

```
        plt.scatter([x0[0]], [x0[1]], color="tab:red", label="Starting Posit
        plt.xlabel("X [m]")
        plt.ylabel("Y [m]")
        plt.title("Drone Position")
        plt.legend(("Trajectory", "Noisy GPS Measurements", "Starting Positi
        plt.show()

    if return_results:
        return Trajectory(x_arr, u_arr, meas_arr)
```

In [10]:
```
data_arrs: list[Trajectory] = []
data_arrs.append(one_trajectory(show=False))
data_arrs.append(one_trajectory(show=False))
data_arrs.append(one_trajectory(return_results=True))
```



Running Kalman Filter

In [11]:
```
mu0 = np.array([1500, 100, 0, 55])
Sigma0 = np.zeros((4,4))
Sigma0[0:2, 0:2] = 250000*np.eye(2)
Sigma0[2:4, 2:4] = np.eye(2)
```

In [12]:
```
mu_arr = np.zeros((N,4))
Sigma_arr = np.zeros((N,4,4))

mu_arr[0] = mu0
```

```
Sigma_arr[0] = Sigma0

x_arr = data_arrs[0].x_arr
u_arr = data_arrs[0].u_arr
meas_arr = data_arrs[0].meas_arr


for t in range(N-1):
    mu_predict = A @ mu_arr[t] + B @ u_arr[t]
    Sigma_predict = A @ Sigma_arr[t] @ A.T + Q

    K = Sigma_predict @ C.T @ inv(C @ Sigma_predict @ C.T + R)

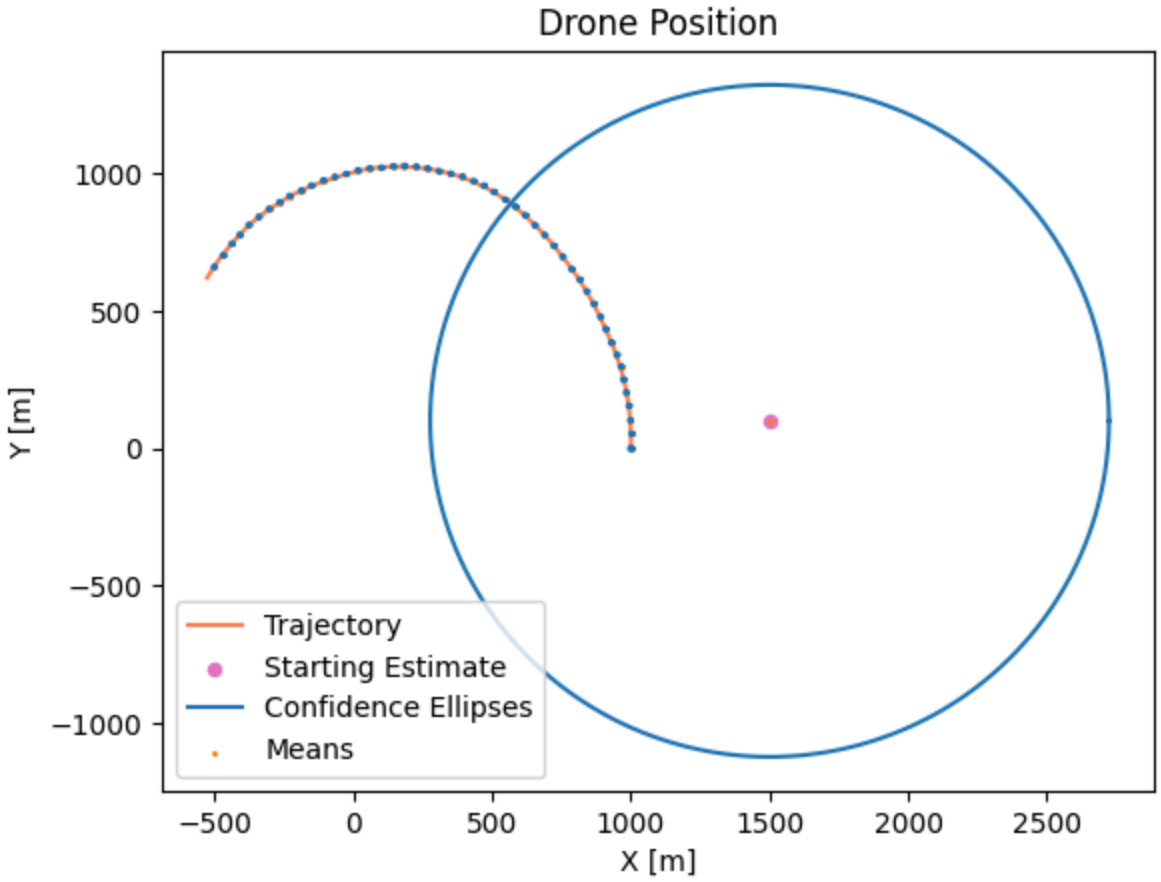
    mu_arr[t+1] = mu_predict + K @ (meas_arr[t] - C @ mu_predict)
    Sigma_arr[t+1] = Sigma_predict - K @ C @ Sigma_predict
```

In [13]:
```
plt.plot(x_arr[:,0], x_arr[:,1], color="coral", zorder=1)
# plt.scatter([x0[0,1]], [x0[0,1]], color="tab:red", label="Starting Positic
plt.scatter([mu_arr[0,0]], [mu_arr[0,1]], color="tab:pink", s=20, marker="o"

for mu, sigma in zip(mu_arr, Sigma_arr):
    mu = mu[0:2]
    sigma = sigma[0:2, 0:2]
    plt.plot(*get_ellipse_points(mu, sigma), color='tab:blue')
    plt.scatter(*mu, color='tab:orange', s=1)

plt.xlabel("X [m]")
plt.ylabel("Y [m]")
plt.title("Drone Position")
plt.legend(("Trajectory", "Starting Estimate", "Confidence Ellipses", "Means
# print([mu_arr[0]])
```

Out[13]: <matplotlib.legend.Legend at 0x1683fcad0>



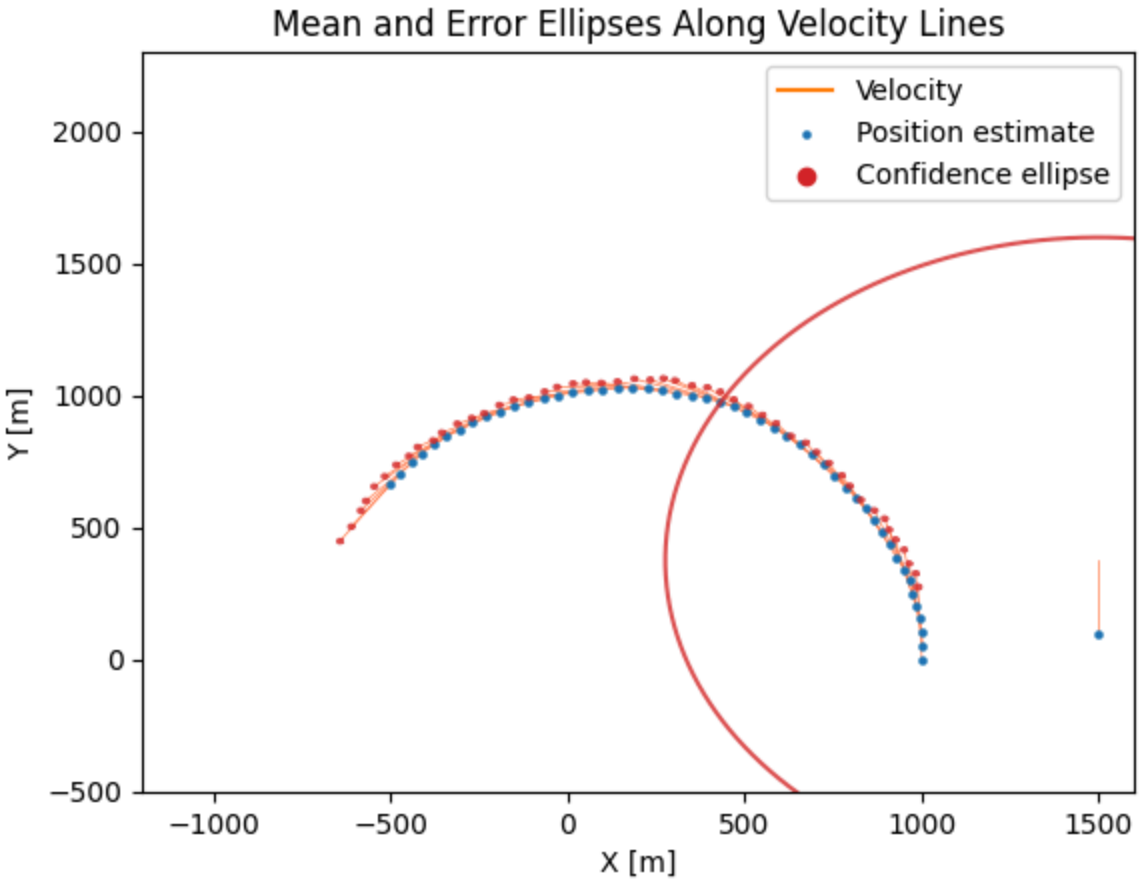
In [14]:
```
handles = []
vel_handle = plt.plot([], [], color="tab:orange", label="Velocity")[0] # jus
pos_handle = plt.scatter([], [], color="tab:blue", s=5, label="Position esti
ell_handle = plt.scatter([], [], color="tab:red", label="Confidence ellipse"

for mu, sigma in zip(mu_arr, Sigma_arr):
    position = mu[0:2]
    velocity = mu[2:4] * 5
    sigma = sigma[0:2, 0:2]
    handles.append(
        plt.plot([position[0], position[0] + velocity[0]], [position[1], pos
                 color="coral", zorder=0, label="Velocity lines", linewidth=
    )
    handles.append(
        plt.scatter([position[0]], [position[1]], color="tab:blue", zorder=1
    )

    ellipse_x, ellipse_y = get_ellipse_points(position, sigma)
    ellipse_x += velocity[0]
    ellipse_y += velocity[1]
    handles.append(
        plt.plot(ellipse_x, ellipse_y, color='tab:red', zorder=2, alpha=0.8,
    )
    plt.xlim(-1200, 1600)
    plt.ylim(-500, -500+2800)

plt.legend(handles=[vel_handle, pos_handle, ell_handle])
plt.title("Mean and Error Ellipses Along Velocity Lines")
```

```
plt.xlabel("X [m]")
plt.ylabel("Y [m]")
plt.show()
```



New Sensor Model

In [15]:
```
R = 9*np.eye(2)

C = np.zeros((2,4))
C[0:2, 2:4] = np.eye(2)

# Kalman
mu0 = np.array([1000, 0, 0, 50])
Sigma0 = np.zeros((4,4))
Sigma0[0:2, 0:2] = np.eye(2)
Sigma0[2:4, 2:4] = np.eye(2)

# R = 100*np.eye(2)
```

In [16]:
```
new_traj = one_trajectory(show=False)

x_arr = new_traj.x_arr
u_arr = new_traj.u_arr
meas_arr = new_traj.meas_arr
```

In [17]:
```
mu_arr = np.zeros((N,4))
Sigma_arr = np.zeros((N,4,4))
```

```
mu_arr[0] = mu0
Sigma_arr[0] = Sigma0

for t in range(N-1):
    mu_predict = A @ mu_arr[t] + B @ u_arr[t]
    Sigma_predict = A @ Sigma_arr[t] @ A.T + Q

    K = Sigma_predict @ C.T @ inv(C @ Sigma_predict @ C.T + R)

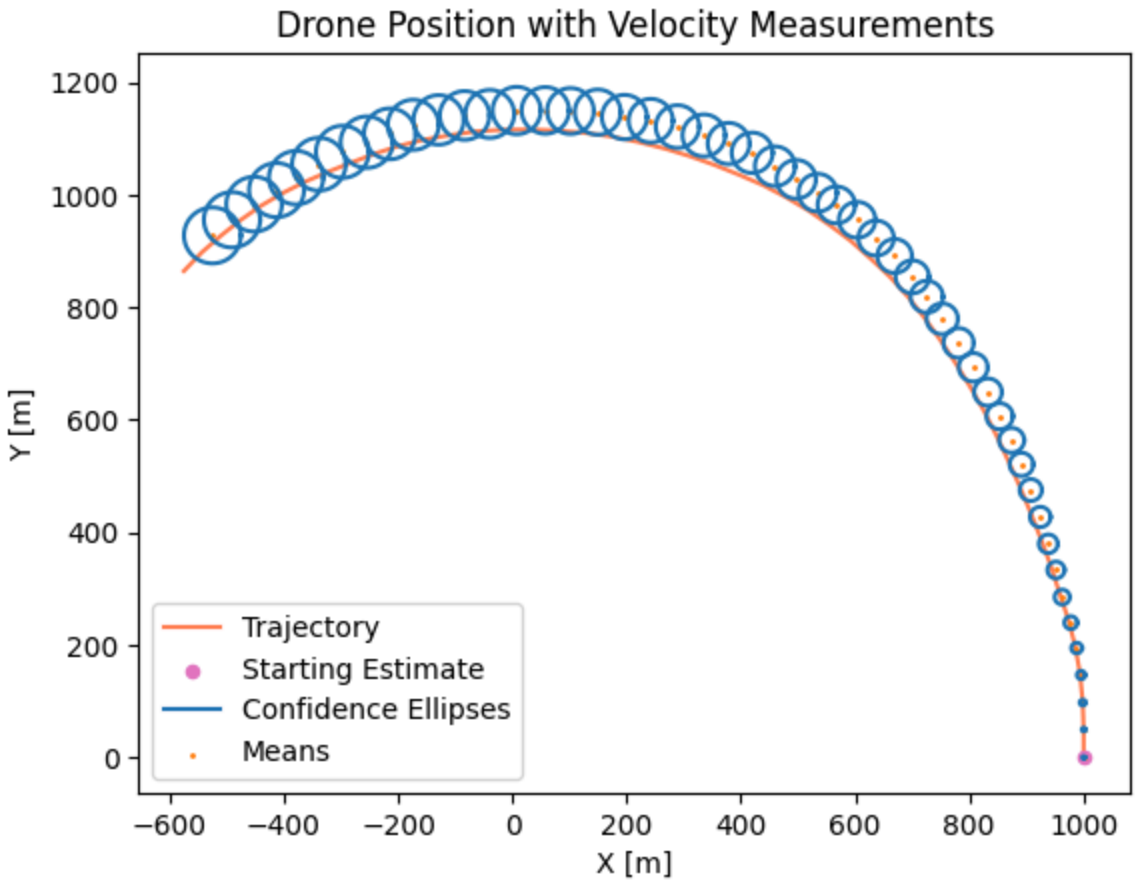
    mu_arr[t+1] = mu_predict + K @ (meas_arr[t] - C @ mu_predict)
    Sigma_arr[t+1] = Sigma_predict - K @ C @ Sigma_predict
```

In [18]:

```
plt.plot(x_arr[:,0], x_arr[:,1], color="coral", zorder=1)
# plt.scatter([x0[0,1]], [x0[0,1]], color="tab:red", label="Starting Positic
plt.scatter([mu_arr[0,0]], [mu_arr[0,1]], color="tab:pink", s=20, marker="o"

for mu, sigma in zip(mu_arr, Sigma_arr):
    mu = mu[0:2]
    sigma = sigma[0:2, 0:2]
    plt.plot(*get_ellipse_points(mu, sigma), color='tab:blue')
    plt.scatter(*mu, color='tab:orange', s=1)

plt.xlabel("X [m]")
plt.ylabel("Y [m]")
plt.title("Drone Position with Velocity Measurements")
plt.legend(("Trajectory", "Starting Estimate", "Confidence Ellipses", "Means
plt.show()
```



In [37]:
```
handles = []
vel_handle = plt.plot([], [], color="tab:orange", label="Velocity")[0] # jus
pos_handle = plt.scatter([], [], color="tab:blue", s=5, label="Position esti
ell_handle = plt.scatter([], [], color="tab:red", label="Confidence ellipse"
traj_handle = plt.plot([], [], color="tab:green", label="Actual Velocity")[0

for mu, sigma in zip(mu_arr, Sigma_arr):
    position = mu[0:2]
    velocity = mu[2:4]
    velocity_sigma = sigma[2:4, 2:4]
    factor = 1
    # plt.plot([position[0], (position[0] + velocity[0]*factor)], [position[
    #             color="coral", zorder=0, label="Velocity lines", linewidth
    plt.scatter([velocity[0]], [velocity[1]], color="tab:blue", zorder=1, s=

    ellipse_x, ellipse_y = get_ellipse_points(velocity, velocity_sigma)
    plt.plot(ellipse_x, ellipse_y, color='tab:red', zorder=2, alpha=0.8, lab

plt.plot(x_arr[:,2], x_arr[:,3], color="tab:green")
plt.legend(handles=[vel_handle, pos_handle, ell_handle, traj_handle])
plt.title("Mean and Error Ellipses with Velocity Measurements")
plt.xlabel("X [m]")
plt.ylabel("Y [m]")
plt.show()
```

